Deep Learning

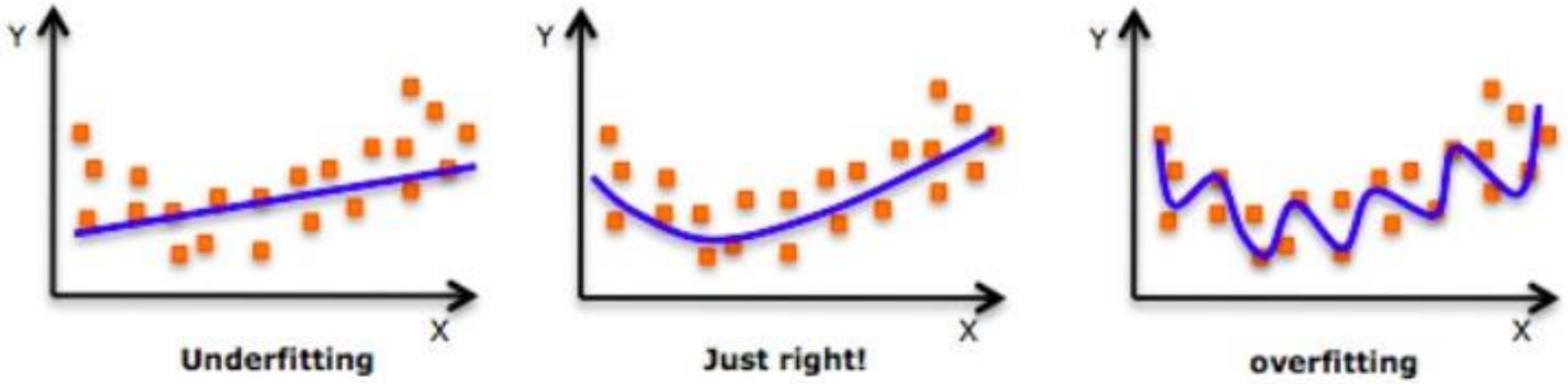
Convolutional Neural Networks-2

Contents

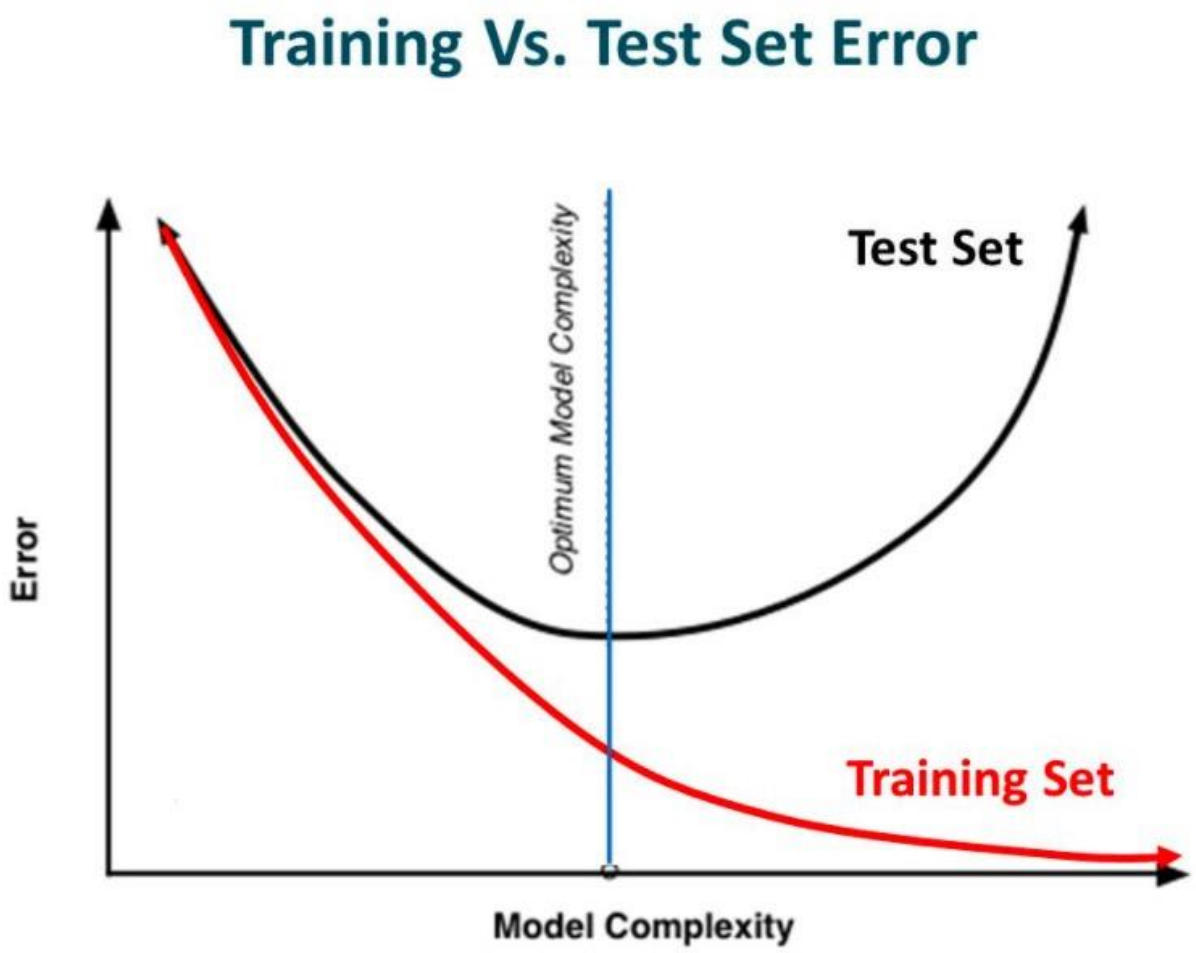
- Regularization Techniques
  - L2 and L1 regularization
  - Dropout
  - Data augmentation
  - Early stopping
- Batch Normalization
- Case Study: AlexNet
- Using Pre-Trained Nets
  - Transfer Learning
  - Fine Tuning

Regularization Techniques

- Objective: Avoiding overfitting



Overfitting



Regularization

- Regularization is a technique which makes slight modifications to the learning
- algorithm such that the model generalizes better.
- This in turn improves the model's performance on the unseen data as well.

Regularization is a technique to discourage the complexity of the model. It does this by penalizing the loss function.

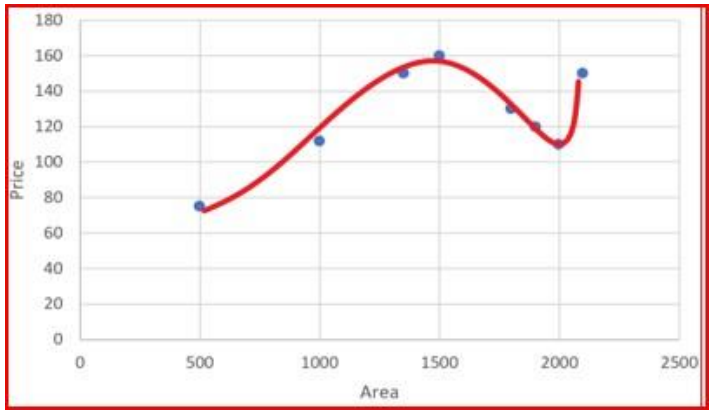
Regularization

- Loss function for a linear regression with 4 input variables:

$$L(x, y) = \sum_{i=1}^{n} (y_i - f(x_i))^2$$

$$f(x_i) = h_\theta x = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3 + \theta_4 x_4^4$$

- As the degree of the input features increases the model becomes complex and tries to fit all the data points:



Regularization

- When we penalize the weights θ3 and θ4 and make them too small, very close to zero. It makes those terms negligible and helps simplify the model

$$f(x_i) = h_\theta x = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3 + \theta_4 x_4^4$$

$$f(x_i) = h_\theta x = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2$$

- Regularization works on assumption that smaller weights generate simpler model and thus helps avoid overfitting.

Regularization

- We add the regularization term to the sum of squared differences between the actual value and predicted value.

- Regularization term keeps the weights small making the model simpler and avoiding overfitting

$$L(x,y) = \sum_{i=1}^{n} (y_i - h_\theta(x_i))^2$$

where $h_\theta x_i = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3 + \theta_4 x_4^4$

$$L(x,y) \equiv \sum_{i=1}^{n} (y_i - h_\theta(x_i))^2 + \lambda \sum_{i=1}^{n} \theta_i^2$$

Regularization

- λ is the penalty term or regularization parameter which determines how much to penalizes the weights.

- When λ is zero then the regularization term becomes zero. We are back to the original Loss function.

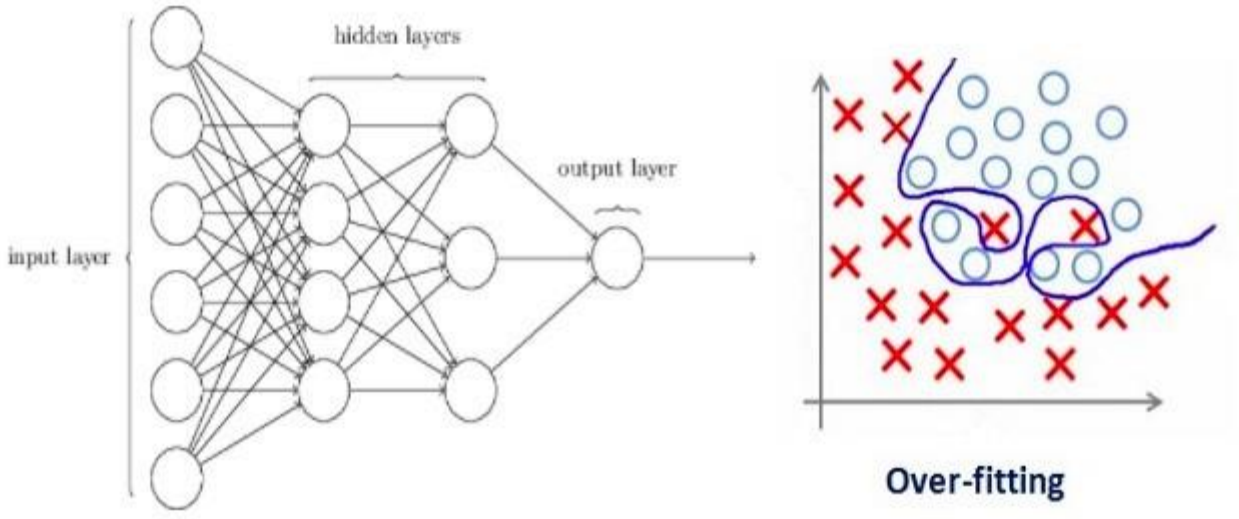
$$L(x, y) = \sum_{i=1}^{n} (y_i - h_\theta(x_i))^2 + 0$$

- When λ is large, we penalizes the weights and they become close to zero. This results in a very simple model having a high bias or is underfitting.

$$h_\theta x = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3 + \theta_4 x_4^4$$

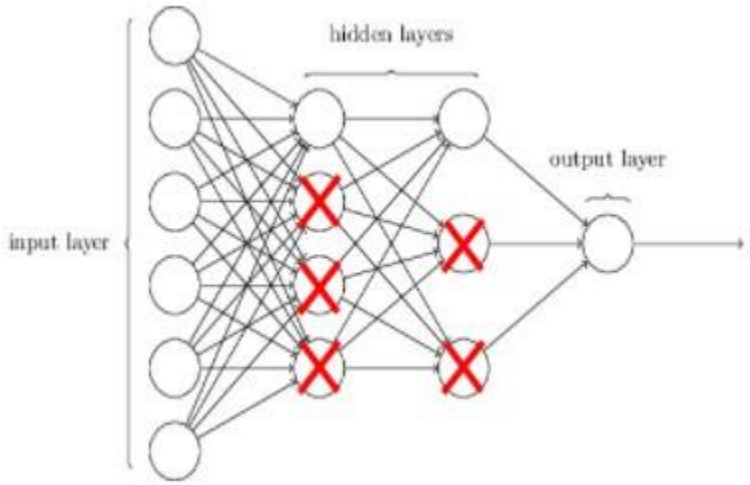
Example - NN

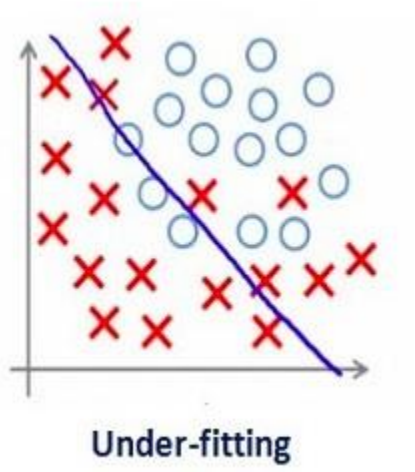
- Consider a neural network which is overfitting on the training data as shown in the image below.



Example - NN

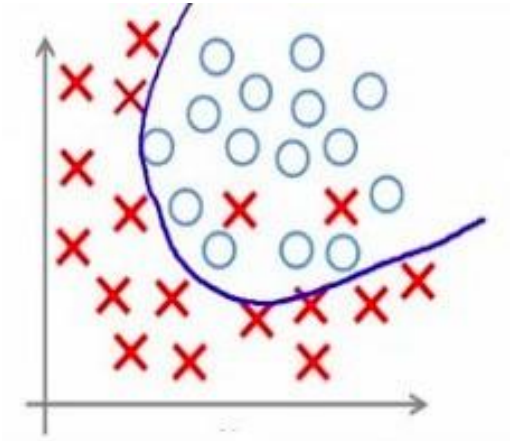
- Regularization penalizes the weight matrices of the nodes.
- Assume that our regularization coefficient is so high that some of the weight matrices are nearly equal to zero.
- This will result in a much simpler linear network and slight underfitting of the training data





Example - NN

- We need to optimize the value of regularization coefficient in order to obtain a well-fitted model



Appropriate-fitting

Regularization Techniques

- L2 and L1 regularization
- Dropout
- Data augmentation
- Early stopping

L1 and L2 Regularization

- L1 and L2 are the most common types of regularization. These update the general cost function by adding another term known as the regularization term.
  - Cost function = Loss (say, binary cross entropy) + Regularization term

- Due to the addition of this regularization term, the values of weight matrices decrease because it assumes that a neural network with smaller weight matrices leads to simpler models.

- Therefore, it will also reduce overfitting to quite an extent.

L1 and L2 Regularization

- L2 Regularization

$$Cost\ Function = Loss + \frac{\lambda}{2} \times \sum \|w\|^2$$

- L1 Regularization

$$Cost\ Function = Loss + \lambda \times \sum |w|$$

- **lambda** is the regularization parameter.
- It is the hyperparameter whose value is optimized for better results.

L2 Regularization – Weight Decay

$$Cost\ Function = Loss + \frac{\lambda}{2} \times \sum \|w\|^2$$

$$J = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(\hat{y}^i, y^i) + \frac{\lambda}{2m} \times \sum \|w\|^2$$

$$J = J_0 + \frac{\lambda}{2m} \times \sum \|w\|^2$$

$$\frac{\partial J}{\partial w} = \frac{\partial J_0}{\partial w} + \frac{\lambda}{m} w$$

$$w = w - \eta \left(\frac{\partial J_0}{\partial w} + \frac{\lambda}{m} w\right)$$

$$w = w - \eta \frac{\lambda}{m} w - \eta \frac{\partial J_0}{\partial w}$$

Original term in GD

$$w = w\left(1 - \eta \frac{\lambda}{m}\right) - \eta \frac{\partial J_0}{\partial w}$$

Weight Decay

L1 Regularization

$$Cost\ Function = Loss + \lambda \times \sum |w|$$

$$\frac{\mathrm{d}}{\mathrm{d}x}|x| = \frac{x}{|x|}$$

$$J = \frac{1}{m}\sum_{i=1}^{m}\mathcal{L}(\hat{y}^i, y^i) + \frac{\lambda}{m} \times \sum |w|$$

$$J = J_0 + \frac{\lambda}{m} \times \sum |w|$$

$$\frac{\partial J}{\partial w} = \frac{\partial J_0}{\partial w} + \frac{\lambda}{m} sign(w)$$

$$w = w\ - \eta\ (\frac{\partial J_0}{\partial w} + \frac{\lambda}{m} sign(w))$$

$$w = w\ - \eta\ \frac{\lambda}{m} sign(w) - \eta\ \frac{\partial J_0}{\partial w}$$

L1 and L2 Regularization

- L2 Regularization

$$w = w - \eta \frac{\lambda}{m} w - \eta \frac{\partial J_0}{\partial w}$$

- L1 Regularization

$$w = w - \eta \frac{\lambda}{m} sign(w) - \eta \frac{\partial J_0}{\partial w}$$

L1 regularization shrinks weights by a constant amount, whereas L2 regularization shrinks weights by an amount proportional to the weights themselves.

L1 and L2 Regularization

```
from keras import regularizers
model.add(Dense(64, input_dim=64,
                kernel_regularizer=regularizers.l2(0.01)
```

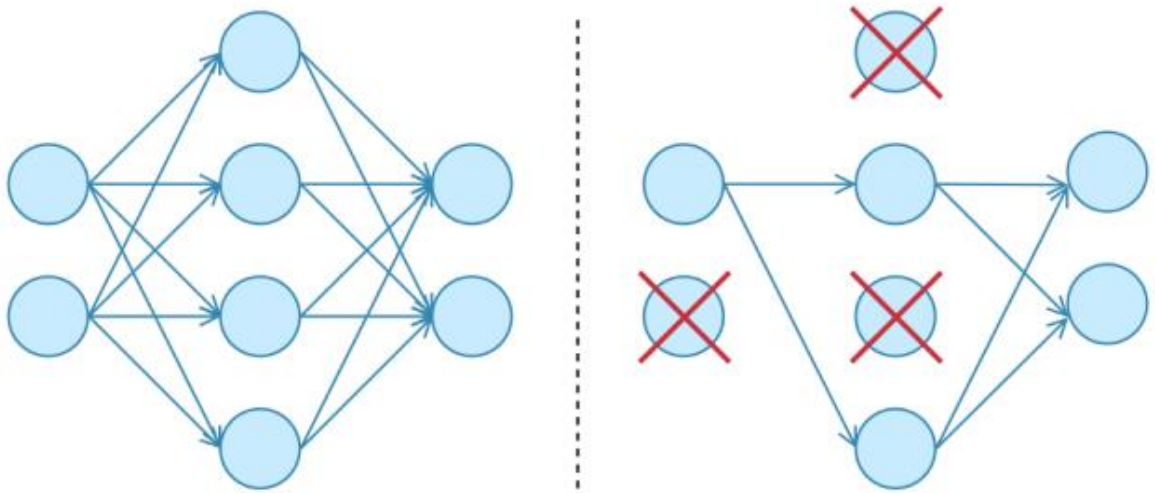
Note: The penalties (regularizes) are applied on a per-layer basis.

Dropout

- Dropout is by far the most popular regularization technique for deep neural networks.
- During training time, at each iteration, a neuron is temporarily “dropped” or disabled with probability p
- This means all the inputs and outputs to this neuron will be disabled at the current iteration.
- The dropped-out neurons are resampled with probability p at every training step, so a dropped out neuron at one step can be active at the next one.
- The hyperparameter p is called the dropout-rate and it’s typically a number around 0.5

Dropout

- Why dropout works?
  - Dropout prevents the network to be too dependent on a small number of neurons, and forces every neuron to be able to operate independently
  - Practically – Parameters linked to dropped neurons are not updated in the iteration it was dropped



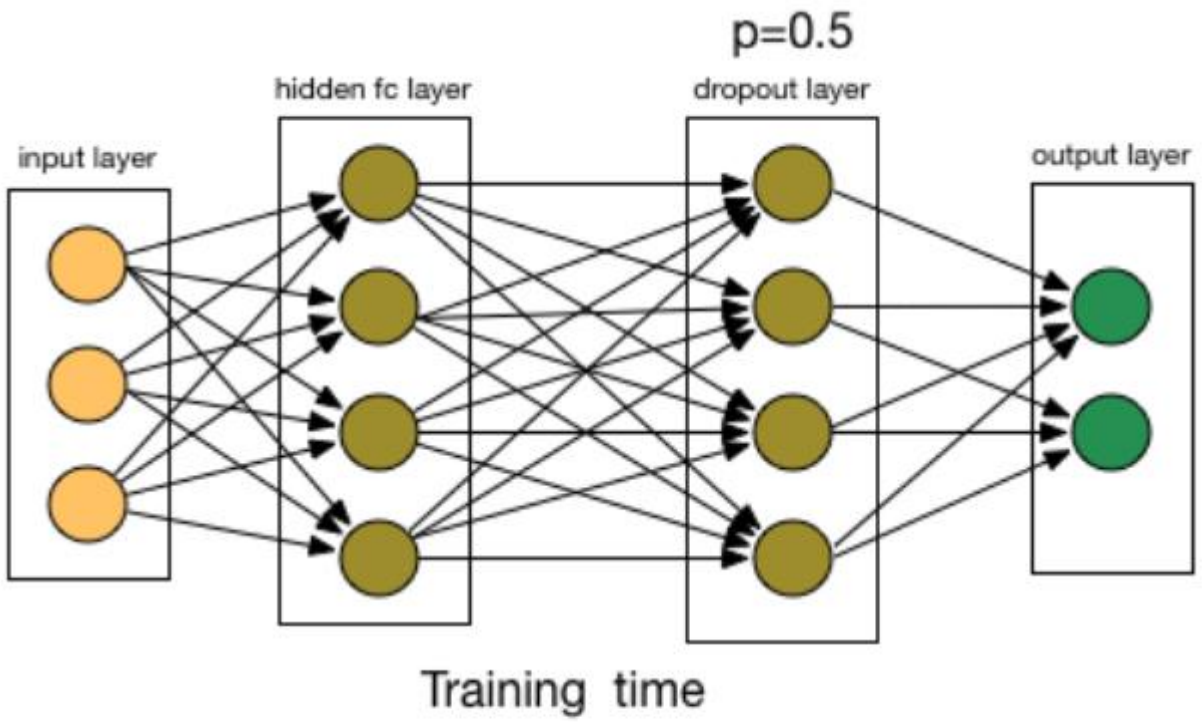
No Dropout

With Dropout

Dropout

- Let’s say you were the only person at your company who knows about finance. If you were guaranteed to be at work every day, your coworkers wouldn’t have an incentive to pick up finance skills.
- But if every morning you tossed a coin to decide whether you will go to work or not, then your coworkers will need to adapt.
- Some days you might not be at work but finance tasks still need to get done, so they can’t rely only at you. Your coworkers will need to learn about finance and this expertise needs to be spread out between various people.
- The workers need to cooperate with several other employees, not with a fixed set of people.
- This makes the company much more resilient overall, increasing the quality and skillset of the employees

Dropout



Note: Dropout is applied to the fully-connected layers of ConvNet only

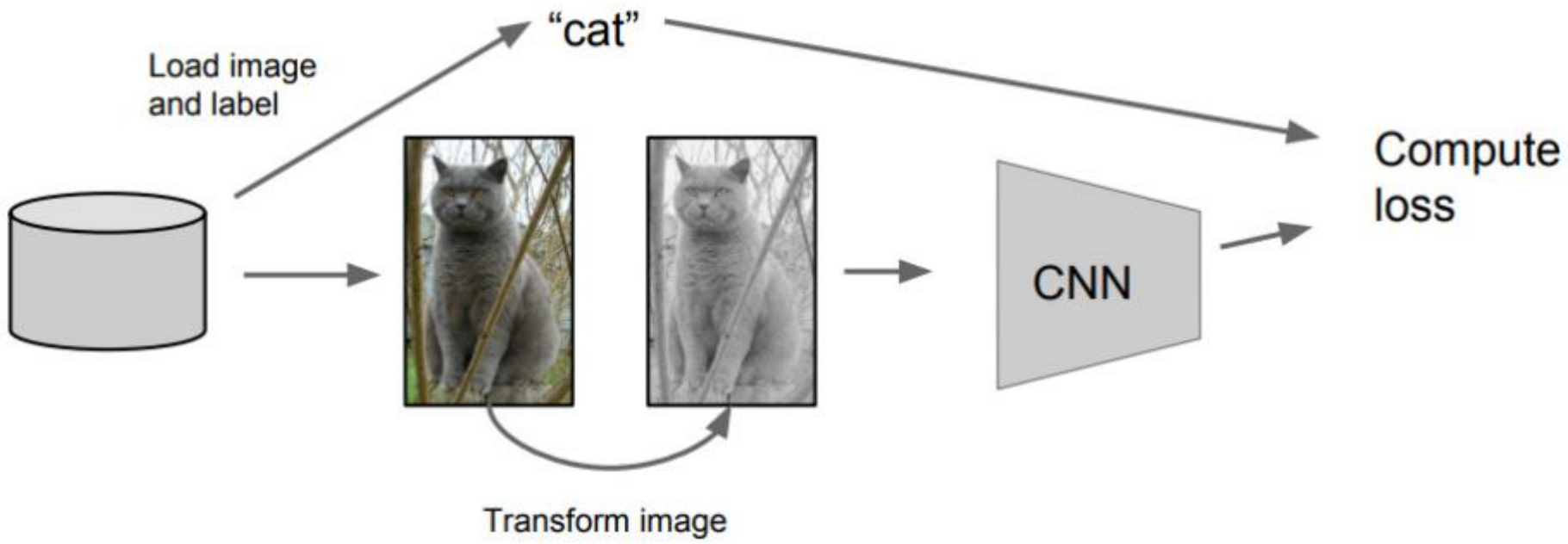
Data Augmentation

- The simplest way to reduce overfitting is to increase the size of the training data.

Data Augmentation Techniques

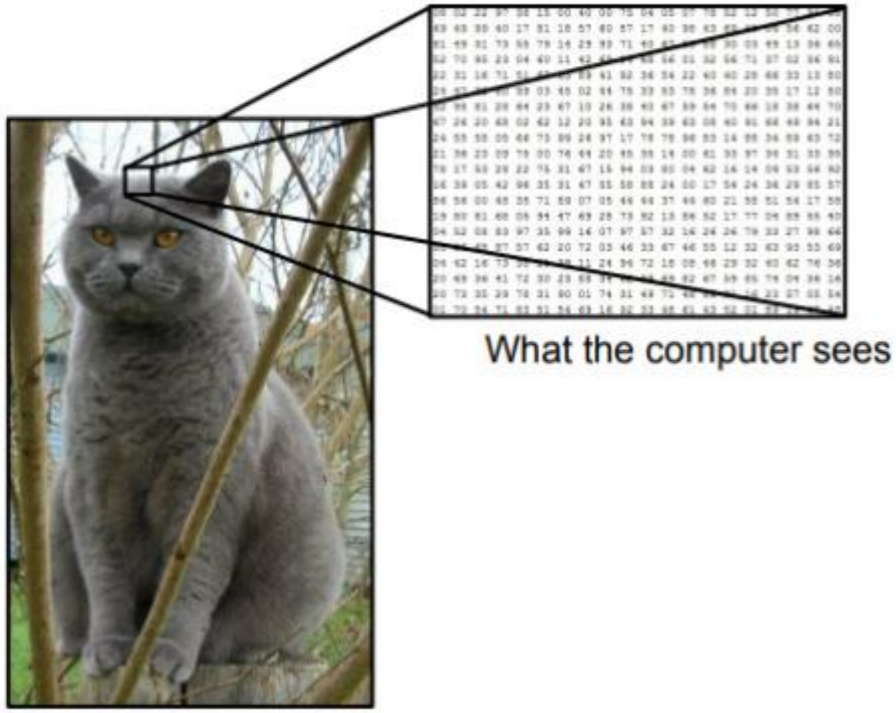
- A computer takes an image as an input, it will take in an array of pixel values.
- Let’s say that the whole image is shifted left by 1 pixel. To us, this change is imperceptible. However, to a computer, this shift can be fairly significant as the classification or label of the image doesn’t change, while the array does.
- Approaches that alter the training data in ways that change the array representation while keeping the label the same are known as data augmentation techniques.
- They are a way to artificially expand the dataset

Data Augmentation



Data Augmentation

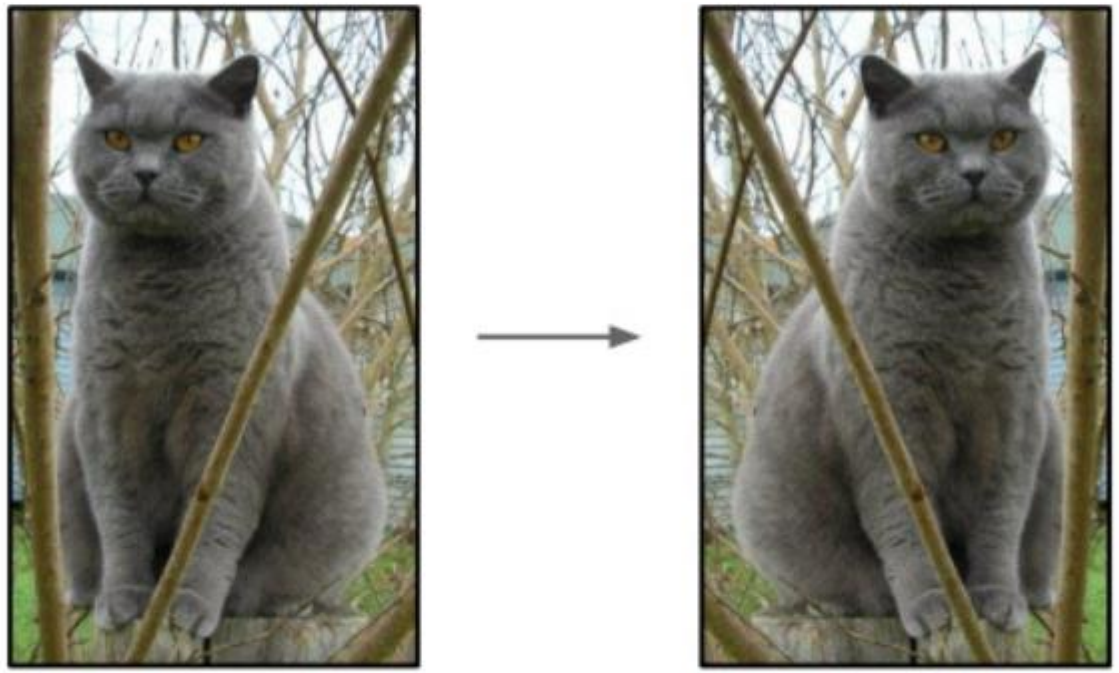
- Change the pixels without changing the label
- Train on transformed data
- VERY widely used



What the computer sees

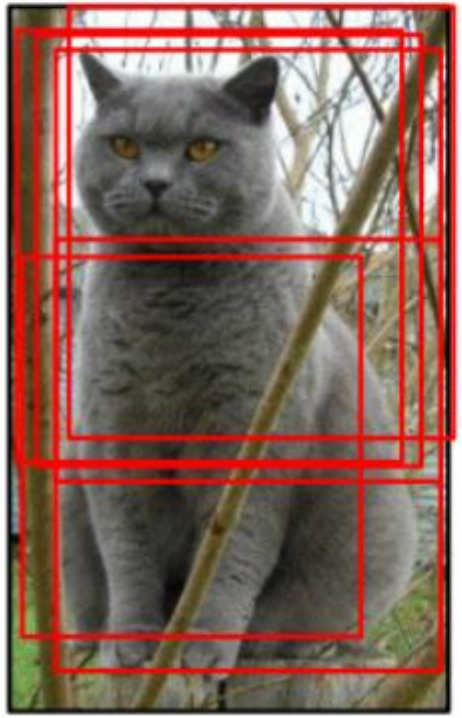
Data Augmentation

- Horizontal Flips



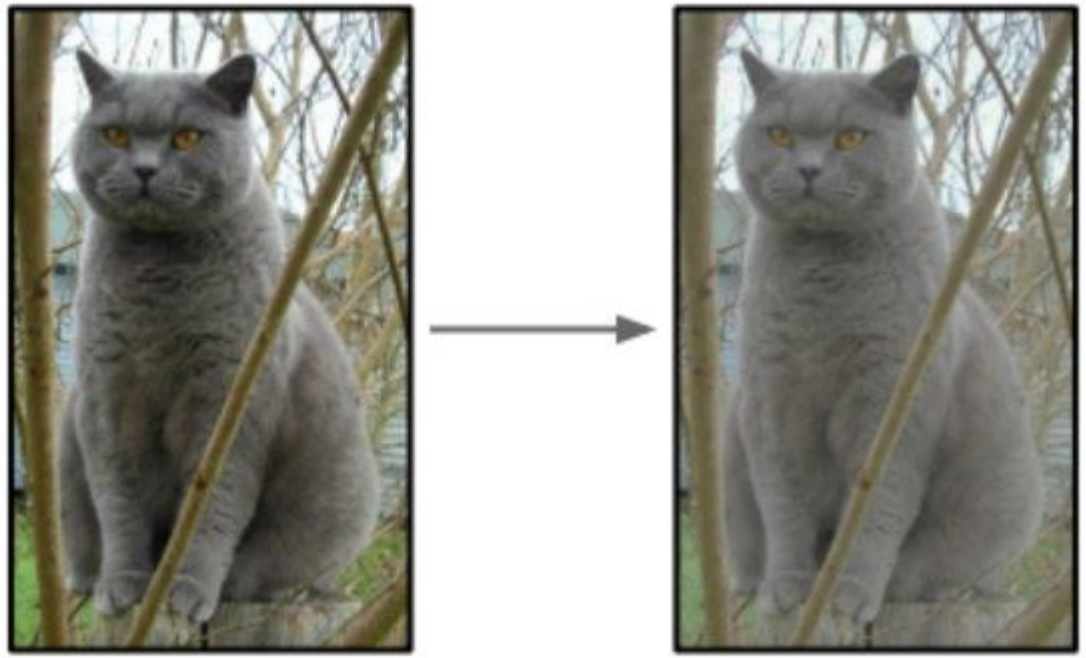
Data Augmentation

- Random Crops/Scales



Data Augmentation

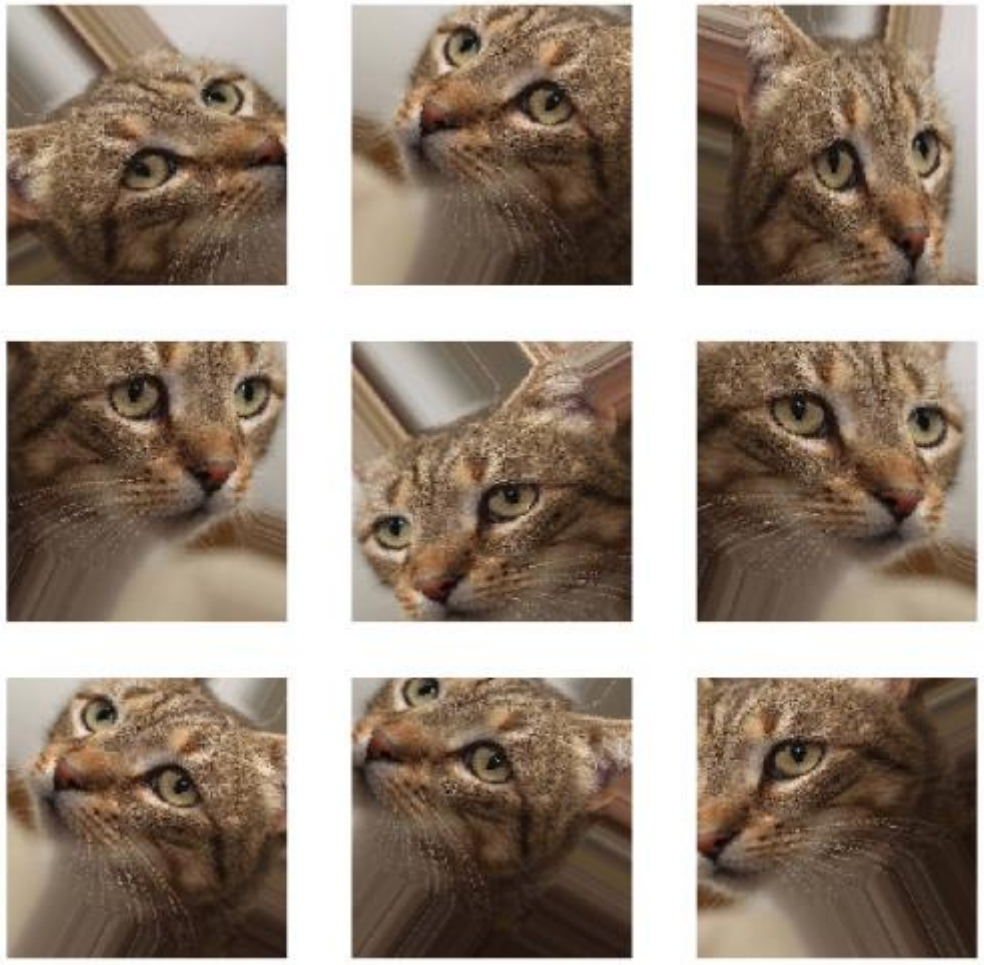
- Color Jitter: Randomly jitter contrast



Data Augmentation

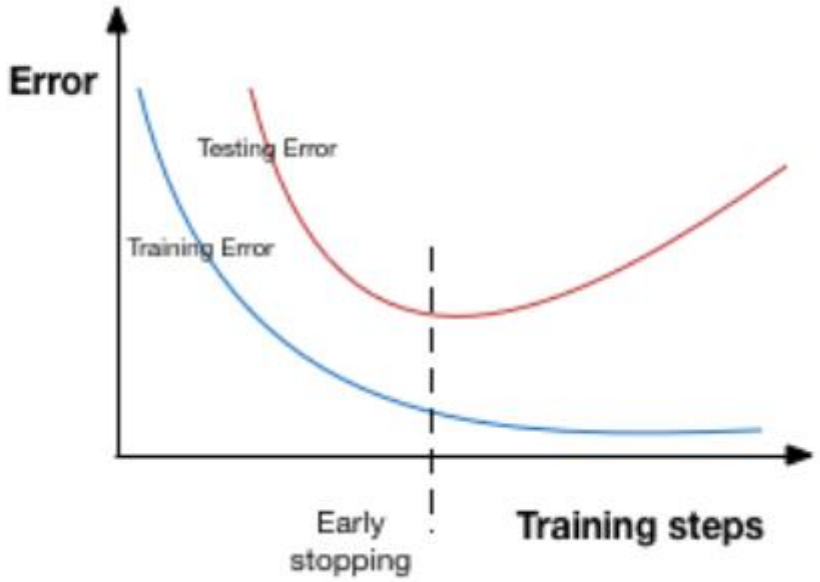
- Random Combinations of:
  - Translation
  - Rotation
  - Stretching
  - Shearing
  - Lens distortion
  - …..

Data Augmentation



Early Stopping

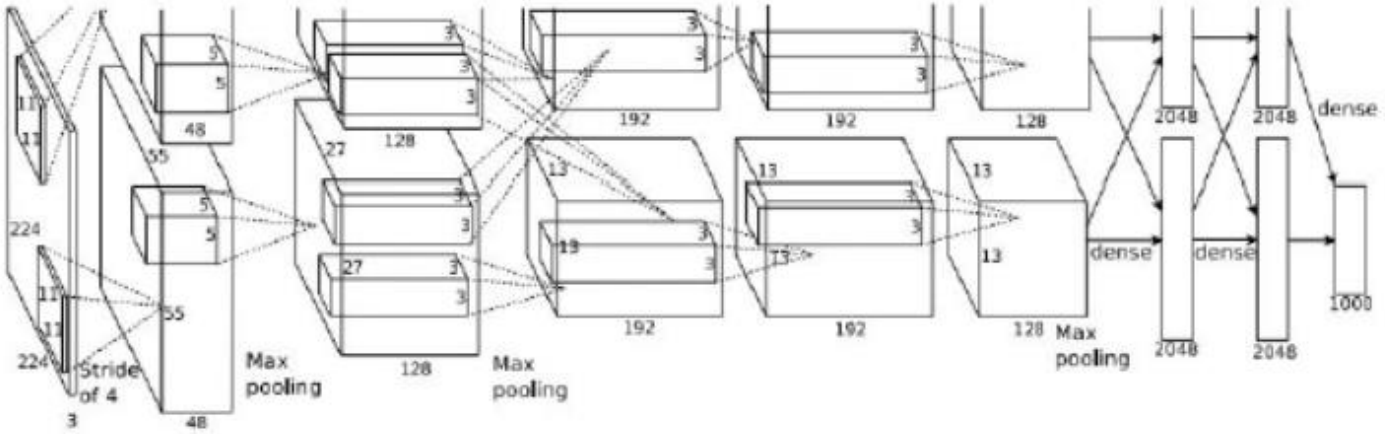
- Early stopping is a kind of cross-validation strategy where we keep one part of the training set as the validation set.
- When we see that the performance on the validation set is getting worse, we immediately stop the training on the model.



Case Study: AlexNet

AlexNet

- Input: 227x227x3 images
- First layer (CONV1): 96 11x11 filters applied at stride 4
- The output volume size?
  - (227-11)/4+1 = 55



AlexNet

- Input: 227x227x3 images
- First layer (CONV1): 96 11x11 filters applied at stride 4
- Output Volume: 55 x 55 x 96
- Total Number of Parameters?
  - Parameters: (11*11*3)*96 = **35K**

AlexNet

- Input: 227x227x3 images
- After CONV1: 55x55x96
- Second layer (POOL1): 3x3 filters applied at stride 2
- Output volume?
  - 27x27x96

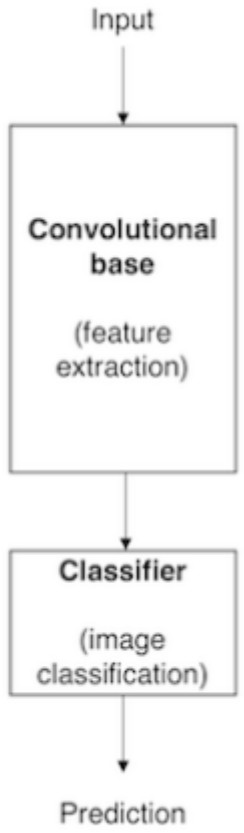
AlexNet

Full (simplified) AlexNet architecture:
[227x227x3] INPUT
[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0
[27x27x96] MAX POOL1: 3x3 filters at stride 2
[27x27x96] NORM1: Normalization layer
[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2
[13x13x256] MAX POOL2: 3x3 filters at stride 2
[13x13x256] NORM2: Normalization layer
[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1
[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1
[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1
[6x6x256] MAX POOL3: 3x3 filters at stride 2
[4096] FC6: 4096 neurons
[4096] FC7: 4096 neurons
[1000] FC8: 1000 neurons (class scores)

Using Pre-Trained ConvNets (Transfer Learning)

Using pre-trained Nets

- A Typical CNN has two parts:
  - **Convolutional base**: which is composed by a stack of convolutional and pooling layers. The main goal of the convolutional base is to generate features from the image.
  - **Classifier**: which is usually composed by fully connected layers. The main goal of the classifier is to classify the image based on the detected features. A fully connected layer is a layer whose neurons have fu connections to all activation in the previous layer.



Using pre-trained Nets

- Deep learning models can automatically learn hierarchical feature representations.
- Features computed by the first layer are general and can be reused in different problem domains, while features computed by the last layer are specific and depend on the chosen dataset and task
- A common misconception in the DL community is that without a Google-esque amount of data, you can't possibly hope to create effective deep learning models.
- While data is a critical part of creating the network, the idea of transfer learning has helped to lessen the data demands.
- **Transfer Learning: Taking a pre-trained model and adapting it to a given problem.**

Using pre-trained Nets

- **Strategy I: Train the entire model**
  - In this case, you use the architecture of the pre-trained model and train it according to your dataset. You're learning the model from scratch, so you'll need a large dataset (and a lot of computational power).

Using pre-trained Nets

- **Strategy II: Fine-Tune a pre-trained model**
  - Change the fully connected layer of the network to match the data under study
  - Continue back propagation and update parameters of all or a subset of layers (Initial layers can be frozen)

Using pre-trained Nets

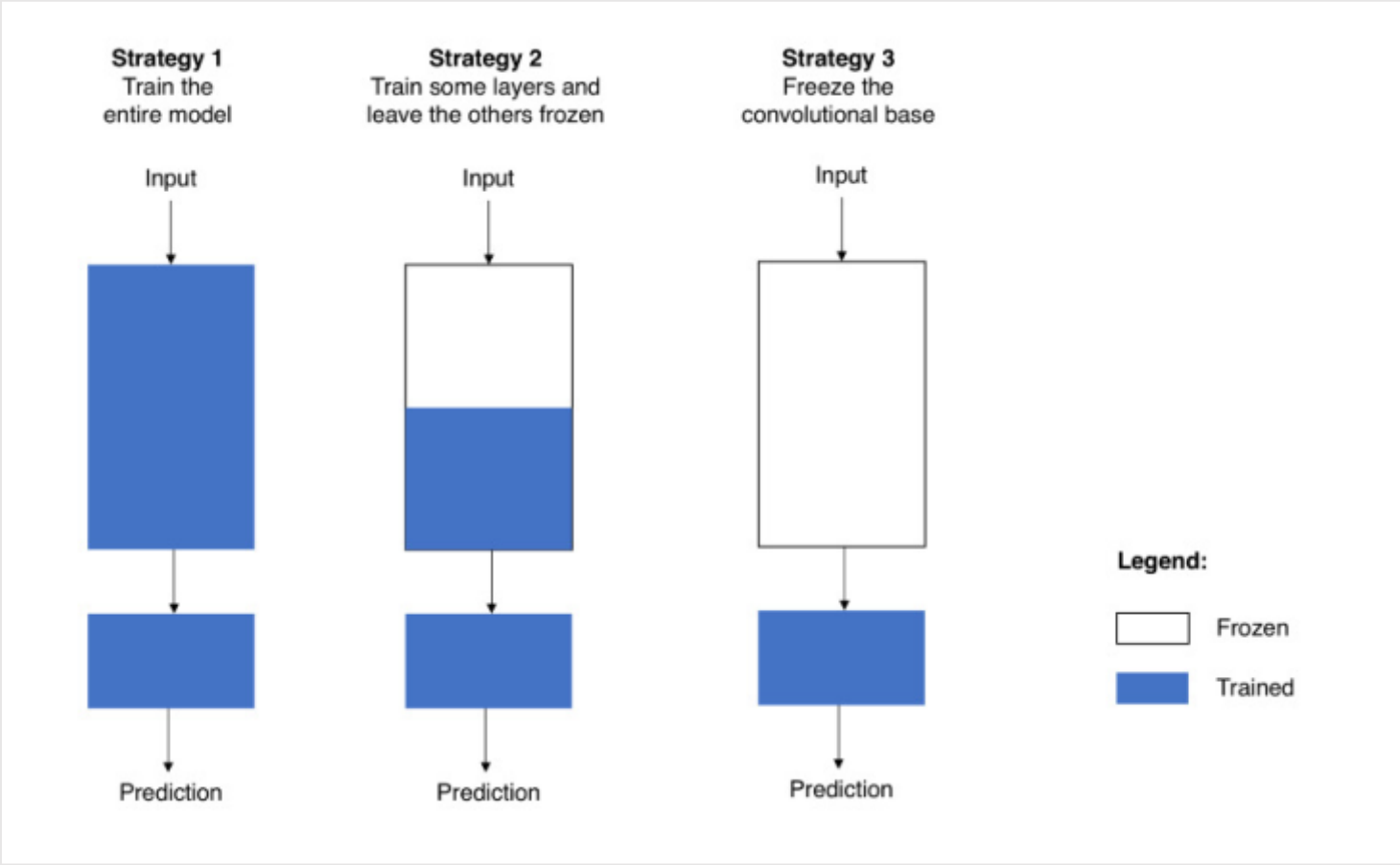
- It is possible to fine-tune all the layers of the ConvNet, or it's possible to keep some of the earlier layers fixed (due to overfitting concerns) and only fine-tune some higher-level portion of the network.

- Earlier features of a ConvNet contain more generic features (e.g. edge detectors or color blob detectors) that should be useful to many tasks, but later layers of the ConvNet becomes progressively more specific to the details of the classes contained in the original dataset.

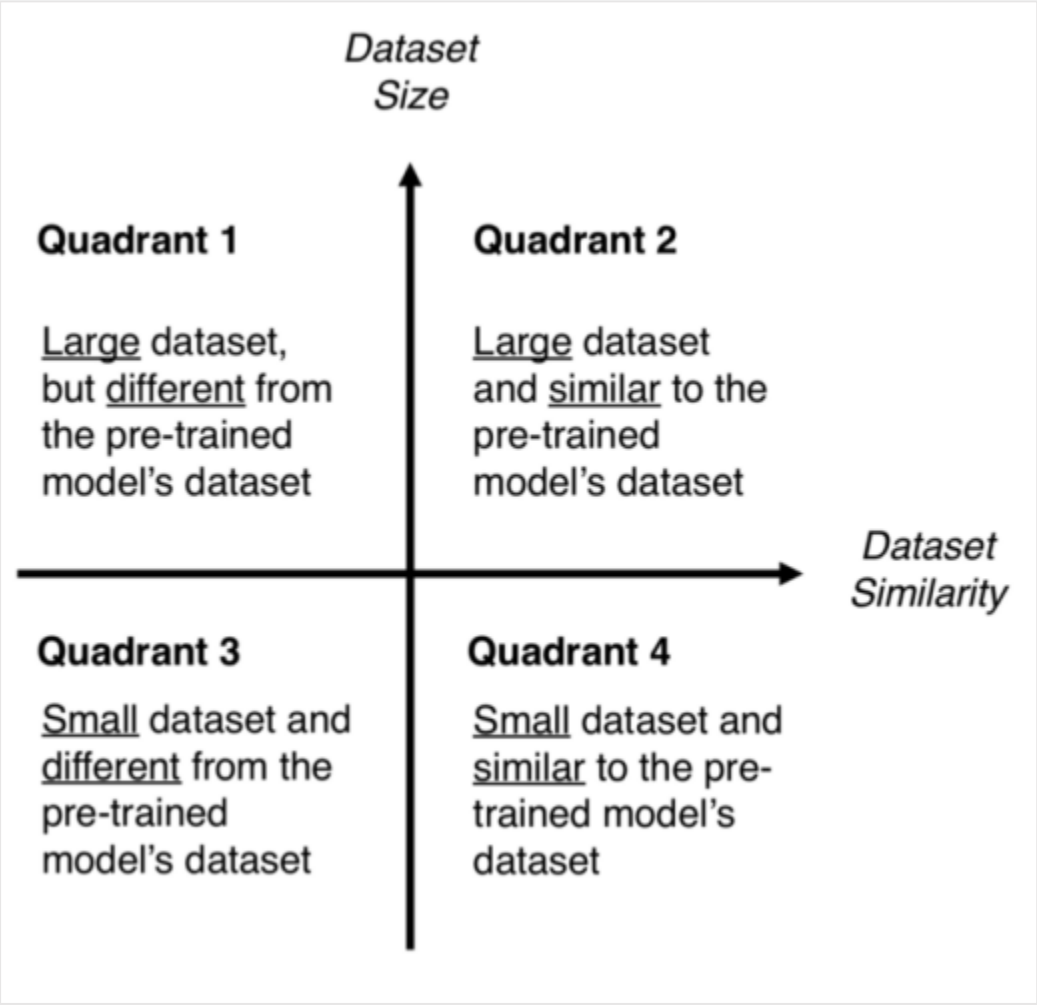
Using pre-trained Nets

- **Strategy III: Use CNN as Feature Extractor**
  - Freeze the convolutional base
  - Pass the data through network and use the output of convolutional base as features
  - Feed features to another classifier
  - **Example:**
    - **For AlexNet:** 4096-D vector for every image that contains the activations of the hidden layer immediately before the classifier.
    - Train Classifier on these features (e.g. SVM)

Summary: Using pre-trained Nets



Using pre-trained Nets: Possibilities



When to use what?